

The debugger

`sprfst debug` runs a program under a real debugger: breakpoints that can be conditional, stepping that understands calls, the call stack, every live variable, watches and memory. It is part of the toolchain, not a separate program, so it needs no build flags and no setup.

```
sprfst debug main.spf
```

The commands

Key	Does
r	run to the first breakpoint
c	continue
s	step into
n	step over (next)
o	step out of this function
b <line>	break at a line
b <line> if <cond>	break only when the condition holds
l	list the breakpoints
d <n>	delete breakpoint number n
k	call stack
v	every variable in the current frame
p <name>	print one variable
w <expr>	watch an expression, shown at every stop
m	memory in use, allocations, collections
q	quit
?	the list above

A session

Take a program with a loop:

```
use std.io

fn total(items: [Int]) -> Int {
  var sum = 0
  for n in items {
    sum = sum + n
  }
  give sum
}
```

```

}

fn main() {
  let numbers = [3, 9, 4]
  let answer = total(numbers)
  io.say("total {answer}")
}

```

Break inside the loop, but only for the large value:

```

(debug) b 6 if n > 3
      break 0 at line 6 if n > 3
(debug) r

      breakpoint dbg.spf:6 in total
      4      var sum = 0
      5      for n in items {
      6 >      sum = sum + n
      7      }
      8      give sum
(debug) v
items      [3, 9, 4]
sum        3
n          9
(debug) k
> 2 total dbg.spf:6
1 main dbg.spf:13
0 <start> dbg.spf:14
(debug) w sum
      watch sum = 3
(debug) c

      breakpoint dbg.spf:6 in total
      watch sum = 12
(debug) q

```

The condition on a breakpoint is ordinary SPRFST, checked in the frame that is stopped, so `n > 3`, `name == "root"` and `items.len() > 100` all work. A condition that cannot be evaluated is treated as false and reported once rather than stopping the program.

Breakpoints land on real lines

Ask to break on a blank line or a comment and the debugger moves the breakpoint forward to the next line that has code, and says so. That is usually what you meant, and it is better than a breakpoint that silently never fires.

Stepping

`s` goes into a call, `n` goes over it, `o` runs until the current function gives back. The listing around the stop always shows the two lines either side, with `>` on the line about to run—not the line that has just finished, which is the usual source of confusion.

Variables you can trust

`v` prints the frame's own locals, in declaration order, with the names you wrote. They survive the optimiser: the debugger runs at `-O0` so the instructions line up with the source exactly. If you want to see what the optimiser did instead, build with `--ir` and read `build/program.spir`.

Memory

`m` reports what the runtime is holding, how many objects are alive and how often the collector has run. It is the quickest way to tell a leak (memory climbing between the same two breakpoints) from a large but stable working set.

When a program crashes

A runtime error prints the same kind of message the compiler does —source line, the value that caused it, and a suggestion —and then the debugger stays open at the point of failure, so `v` and `k` still work.

Exercise

Put a deliberate off by one into the `total` function above, break on the line that gives the answer with a condition that only holds when the sum is wrong, and find it with `k` and `v`.